



TATA
CONSULTANCY
SERVICES

DATA ENGINEER

Advanced Scenario Based Interview Q&A (3–5 Years)...

1. Your Spark job is slow only on some days although data volume is same. What will you check?

Answer:

First, I would confirm whether the **input data distribution** is the same.

Even if the volume is same, the **data skew** can change daily. For example, one day 70% of records belong to one customer_id, which sends most data to a single partition.

Steps I follow:

1. Check Spark UI → **Stages** → **Shuffle Read/Write size**
2. Look for tasks running much longer than others
3. Check partition size using:

```
df.rdd.getNumPartitions()
```

4. Repartition or use salting if skew exists

Other checks:

- Executor memory usage (GC time)
- External dependency latency (API, DB)



- Cluster auto-scaling behavior
- File size imbalance in source

In production, I once fixed this by:

- Adding **salting key**
- Increasing shuffle partitions
- Enabling adaptive query execution (AQE)

2. A production pipeline failed and business wants data in 30 minutes. What will you do?

Answer:

I follow **three parallel tracks**:

1. Business communication

Inform stakeholders:

“Issue identified, ETA 25 minutes, working on workaround.”

This builds trust.

2. Technical triage

I immediately check:

- Orchestration logs (ADF/Airflow)
- Spark job logs
- Last successful run inputs

Common fast fixes:

- Rerun only failed partition
- Disable bad transformation
- Use last day backup data
- Run partial pipeline

3. Root cause prevention

After delivery:

- Add retry logic
- Improve alerting
- Add data quality validation
- Write RCA document

TCS values engineers who balance **delivery + stability**.



3. How do you design a data pipeline for 10 TB daily data?

Answer:

I design in **layers**:

Ingestion

- Kafka / Event Hub / Batch SFTP
- Use partitioned files (hour/date)

Storage

- Data Lake (ADLS / S3)
- Folder structure:

```
/raw/source/date=YYYY-MM-DD/  
/processed/  
/curated/
```

Processing

- Spark incremental loads
- Partition by date + business key

Optimization

- Use Parquet/Delta
- Partition pruning
- Z-ordering (Delta)
- Avoid small files

Monitoring

- Row count checks
- Null checks
- SLA alerts

This ensures **scalability, cost control, and recoverability**.

4. Difference between repartition and coalesce? When used?

Answer:

Aspect	Repartition	Coalesce
--------	-------------	----------



Shuffle	Yes	No
Performance	Slower	Faster
Use case	Increase partitions	Reduce partitions

Example:

After filtering large dataset:

```
df_filtered.coalesce(10)
```

Before join:

```
df.repartition(200)
```

In TCS projects, we use:

- repartition before heavy joins
- coalesce before writing output

5. How do you handle late-arriving data?

Answer:

I use:

- Event time
- Watermarking
- Delta merge logic

Example:

If sales data comes late:

1. Store raw data
2. Run daily merge job:

```
MERGE INTO fact_sales t
USING staging_sales s
ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE
WHEN NOT MATCHED THEN INSERT
```



Also maintain:

- Late arrival window (e.g., 7 days)
- Audit columns

6. How do you optimize Spark joins?

Answer:

Steps:

1. Broadcast small table:

```
broadcast(df_small)
```

2. Use same partition key
3. Increase shuffle partitions
4. Avoid UDFs
5. Use bucketing for repeated joins
6. Enable AQE

In one project, broadcast join reduced job time from **45 min to 8 min**.

7. Explain slowly changing dimension (SCD) with scenario.

Answer:

Customer changes address.

Type 1

Overwrite (no history)

Type 2

Create new record:

```
| customer_id | address | start_date | end_date | current |
```

Used for compliance and reporting.

We usually implement using:

- surrogate keys
- merge logic
- current_flag

8. How do you handle schema changes in production?

Answer:

I:

- Enable schema evolution (Delta)
- Validate schema before load
- Maintain versioned tables
- Alert on breaking changes

Never auto-apply in curated layer without validation.

9. Explain data skew and how you solved it.

Answer:

In telecom project:

- 60% data belonged to one customer

Fix:

- Add salt key:

```
df.withColumn("salt", rand()*10)
```

- Join using salt

Result: job reduced from **3 hours to 35 minutes**

10. Difference between fact and dimension table?

Answer:

Fact:

- Transactions
- Large
- Numeric measures

Dimension:

- Descriptive
- Smaller
- Used for slicing

Example:

Fact: Sales

Dimension: Customer, Product, Date



11. How do you ensure data quality?

Answer:

I implement:

- Null checks
- Duplicate checks
- Range checks
- Row count reconciliation
- Great Expectations / custom framework
- Quarantine bad records

12. Explain partitioning vs bucketing.

Answer:

Partitioning:

- Directory based
- Used for filtering

Bucketing:

- Hash based
- Used for joins

Use partition on date

Use bucket on customer_id

13. What is idempotent pipeline?

Answer:

Running pipeline multiple times gives same result.

Achieved by:

- Using merge instead of insert
- Deduplication
- Primary keys

Critical for reruns.

14. How do you manage memory issues in Spark?

Answer:

- Cache selectively



- Avoid collect()
- Increase executor memory
- Use Kryo serialization
- Tune partitions

15. How do you design disaster recovery?

Answer:

- Multi-region storage
- Daily snapshots
- Metadata backup
- Replay from raw layer
- Infra as code

16. Explain star vs snowflake schema.

Answer:

Star: simple, fast queries

Snowflake: normalized, less storage

TCS prefers **star for reporting**, snowflake for complex domains.

17. How do you tune SQL queries?

Answer:

- Indexing
- Partition pruning
- Avoid SELECT *
- Use CTEs
- Analyze execution plan

18. How do you manage incremental loads?

Answer:

- Watermark column
- CDC
- Merge logic
- Audit table

19. Explain CAP theorem in data systems.

Answer:

Choose 2 of:



- Consistency
- Availability
- Partition tolerance

Kafka = AP
RDBMS = CA

20. How do you monitor pipelines?

Answer:

- SLA dashboards
- Failure alerts
- Data drift alerts
- Cost monitoring
- Log analytics

